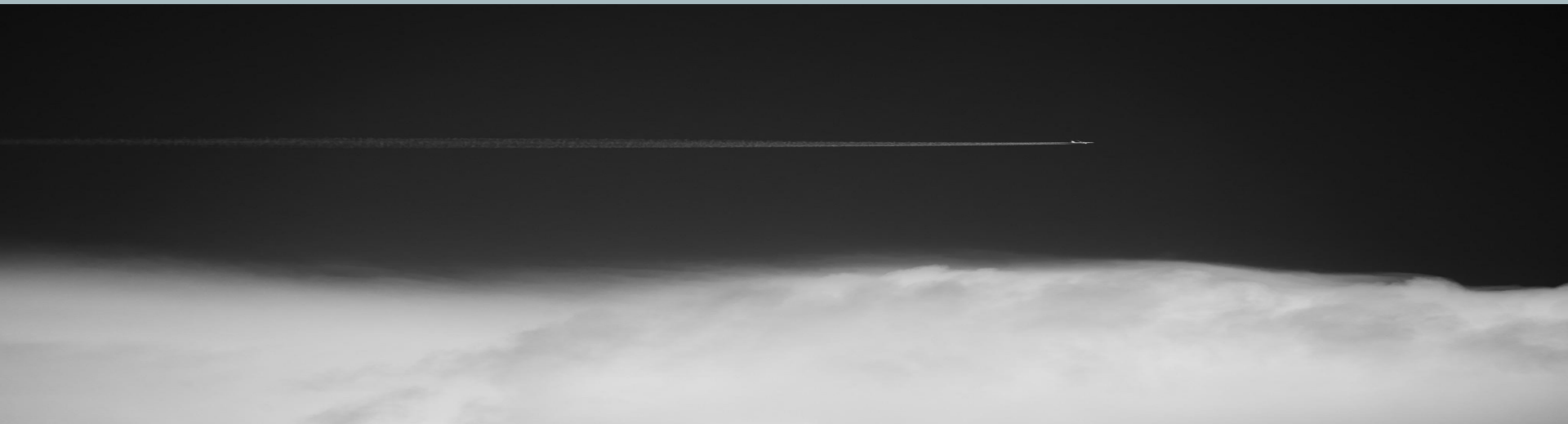


Flight Tracker



Rianna Armour | Theebika Iyer | Omar Usmani | Kyle Yannelli | William Chien

Team 4

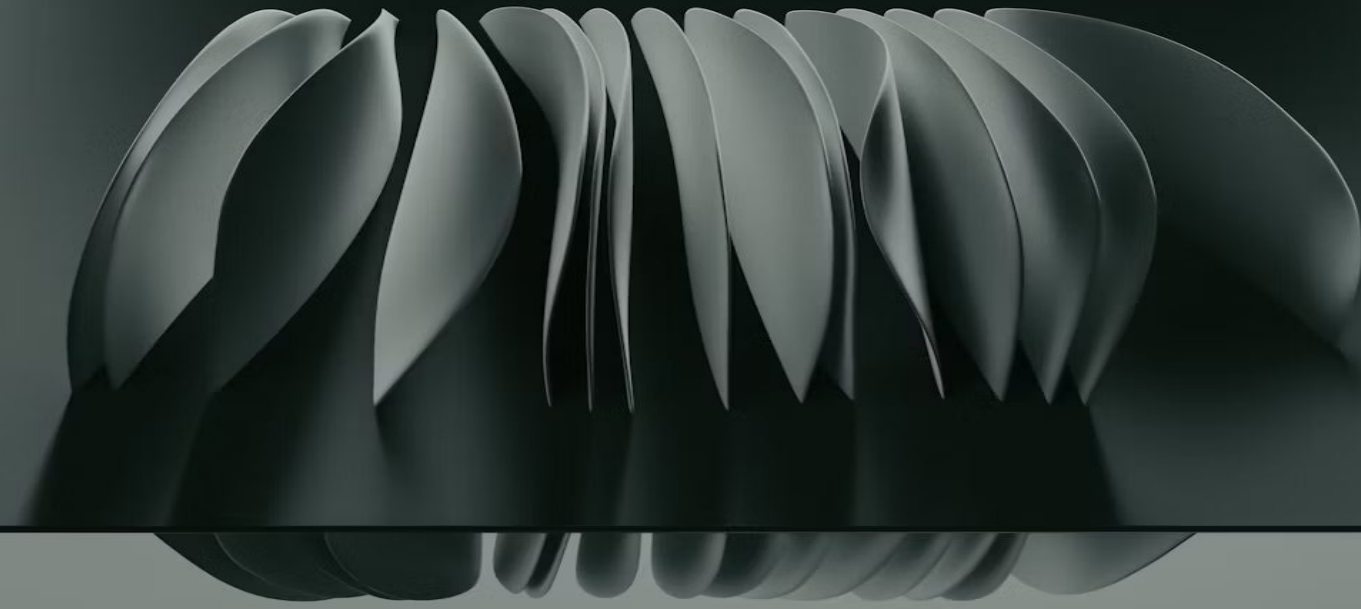
01 Introduction

02 ERD

03 Application

04 Lessons Learned

Introduction



Introduction

Problem:

With air travel becoming more common than ever, the need for an efficient way to track flights and ensure travelers are well-informed is growing

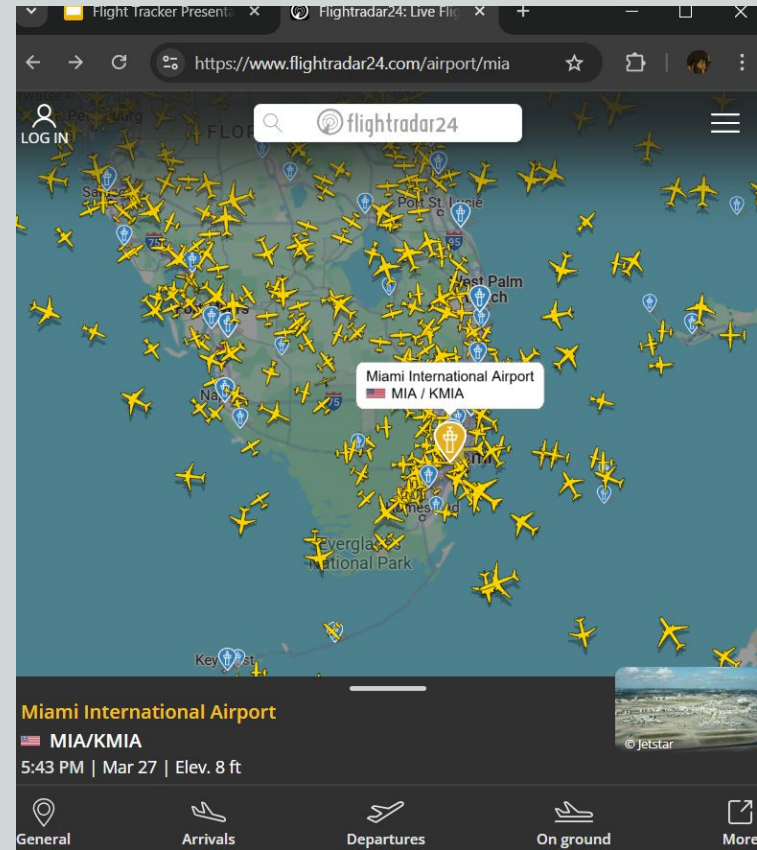
Solution:

A simple, seamless, and user-friendly flight tracking tool.

Introduction Cont'd

The most popular flight tracking applications, such as Flightradar24 and FlightAware, do not rely on the Spring Framework, or MySQL as their database.

This project uses the Spring Framework and MySQL to implement flight tracking features.



Technology

MySQL



Svelte



Spring



aviationstack



Leaflet





Diagrams

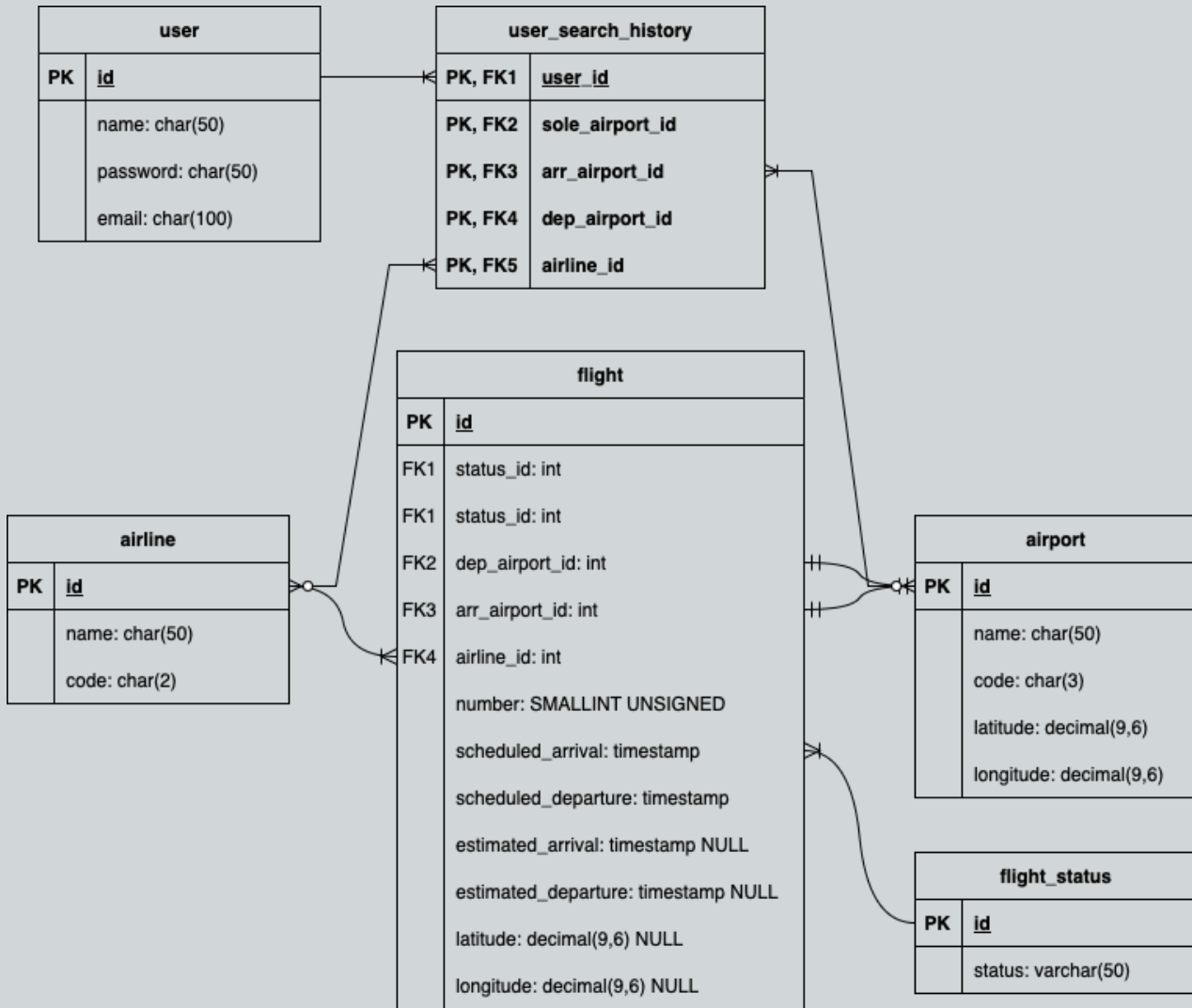
ERD

Airline has many flights
Airport has many flights
User has many search histories

A flight will always contain its status,
departing airport, arrival airport, airline,
flight number, scheduled arrival, and
scheduled departure.

A flight may or may not have the
estimated departure, estimated arrival,
latitude, and longitude.

A flight does not use the flight number
for a primary key as it can be reused.
Even the IATA number, such as DL0012,
which is the airline code combined with
the flight number.





Application

Features



Unregistered Users

- Search for flights by departing airport, arrival airport, and airline.
- Find a flight from its IATA Flight Number.
- Check the status of a flight.

Registered Users

- Everything an unregistered user can do.
- Track in-air flights on a map (live view).
- View search history for flights.

Quality Testing our Application

- ❖ We implemented tests to make sure our Application and its features were performing as intended.

Login & Security:

1. Makes sure users can log in safely with their username and password
2. Checks that passwords are stored securely in the system

Flight Details:

1. Confirms we can look up airlines by their code (like "AA" for American Airlines)
2. Makes sure we can find airports using their 3-letter codes
3. Checks if flight status updates (like "Delayed" or "On Time") work correctly

Finding Flights:

1. Tests if we can see all available flights
2. Checks if we can filter flights by specific airlines
3. Makes sure we can find flights going to or from specific airports

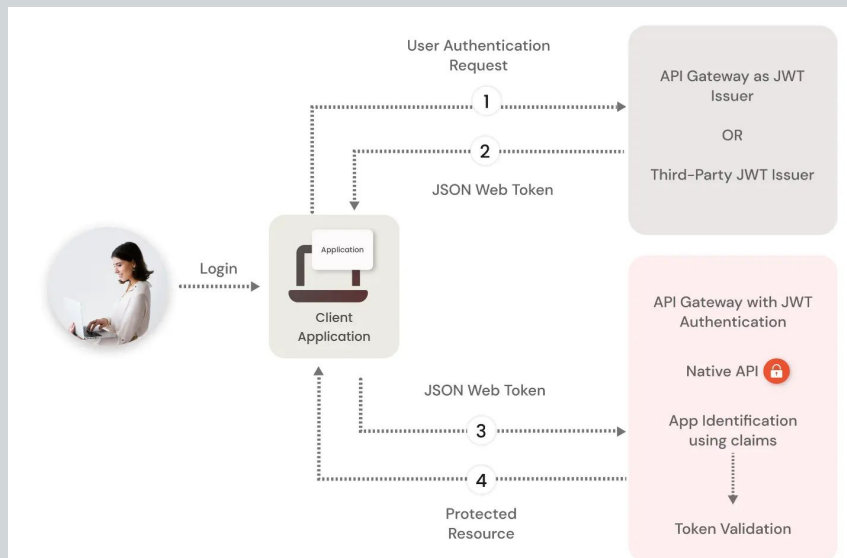
User Features:

1. Verifies that the system saves what flights users search for
2. Checks if users can see their past flight searches

User Security

Implemented a simple security process that ensures proper authentication:

1. **User is created:** The user signs up, and their information is securely stored.
2. **Login and Token Creation:** After logging in, a unique token is generated to confirm their identity.
3. **Secure Access:** The token is used for future requests, allowing the system to verify the user.



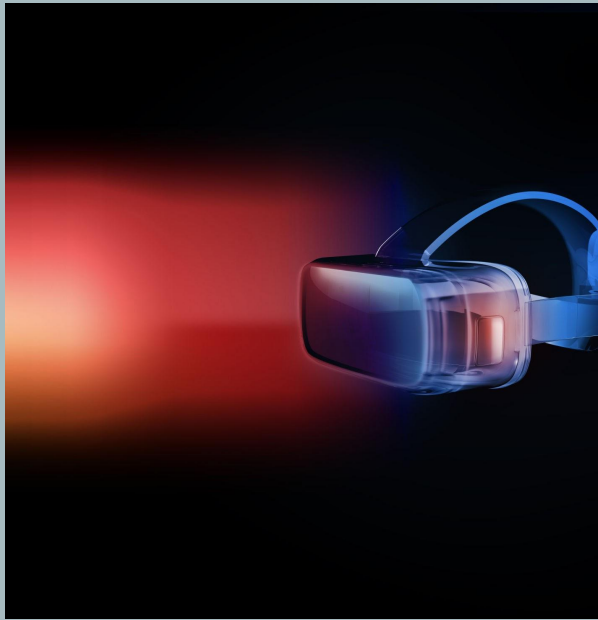
Stretch Goals:

Initial goals:

- **Implement Live Map:** Creating a live view of a planes location on a map
- **Adding Flight Path to the Map:** A path that shows the start-end of a flight during travel
- **Making Frontend Aesthetic:** Making the pages user-friendly, easy to navigate
- **Future Flights:** Get upcoming flights for future dates, potentially 3rd party API limited

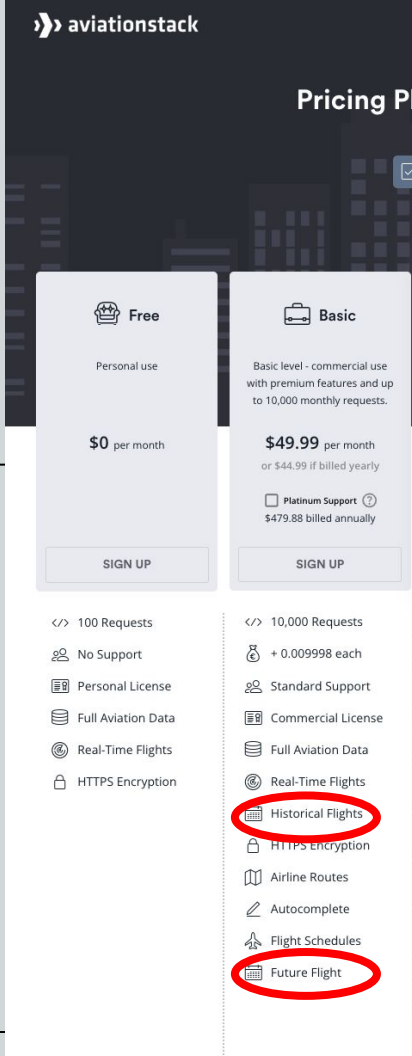
Which stretch goals were accomplished?

Live Demos



Future Flights?

AviationStack's documentation is misleading...



The image shows a pricing comparison table for AviationStack. It compares a 'Free' plan with a 'Basic' plan. The 'Free' plan is for personal use, costs \$0 per month, and includes 100 requests, no support, a personal license, full aviation data, real-time flights, and HTTPS encryption. The 'Basic' plan is for basic level commercial use, costs \$49.99 per month (or \$44.99 if billed yearly), and includes 10,000 requests, 0.009998 requests per second, standard support, a commercial license, full aviation data, real-time flights, historical flights, HTTPS encryption, airline routes, autocomplete, flight schedules, and future flight. Both plans have a 'SIGN UP' button.

Free	Basic
Personal use	Basic level - commercial use with premium features and up to 10,000 monthly requests.
\$0 per month	\$49.99 per month or \$44.99 if billed yearly
SIGN UP	SIGN UP
</> 100 Requests	</> 10,000 Requests
🛡️ No Support	🕒 + 0.009998 each
📄 Personal License	🛡️ Standard Support
📄 Full Aviation Data	📄 Commercial License
🕒 Real-Time Flights	📄 Full Aviation Data
🔒 HTTPS Encryption	🕒 Real-Time Flights
	📄 Historical Flights
	🔒 HTTPS Encryption
	📖 Airline Routes
	✍️ Autocomplete
	🛫 Flight Schedules
	📅 Future Flight

API Endpoints

Flights

Real-Time Flights Available on: All plans

The API is capable of tracking flights and retrieving flight status information in real-time. In order to look up real-time info about one or multiple flights, you can use the API's **flights** endpoint together with optional parameters to filter your re

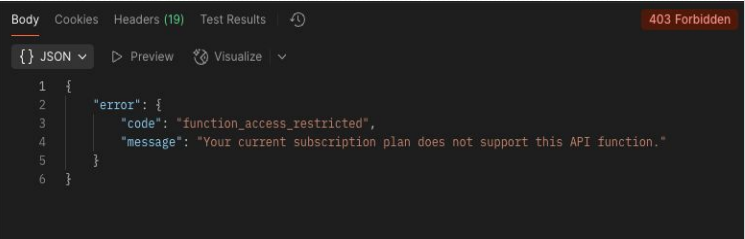
Example API Request:

```
https://api.aviationstack.com/v1/flights  
? access_key = YOUR_ACCESS_KEY
```

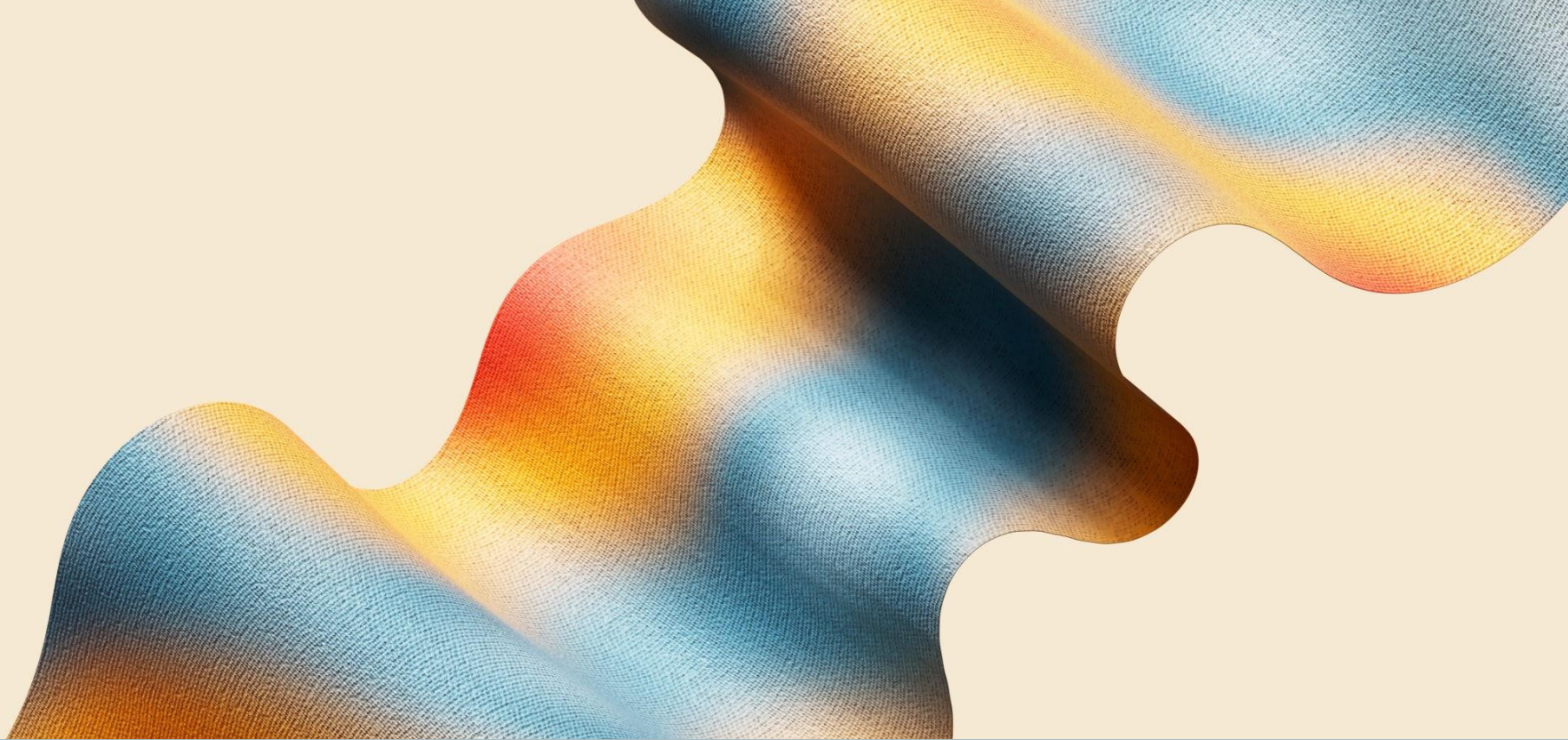
Sign Up to Run API

HTTP GET Request Parameters:

Object	Description
flight_date	[Optional] Filter your results by providing a flight date in the format YYYY-MM-DD. Ex: 2019-02-28
dep_iata	[Optional] Filter your results by departure city or airport using an IATA code. You can use IATA codes using the Airports or Cities API endpoints.



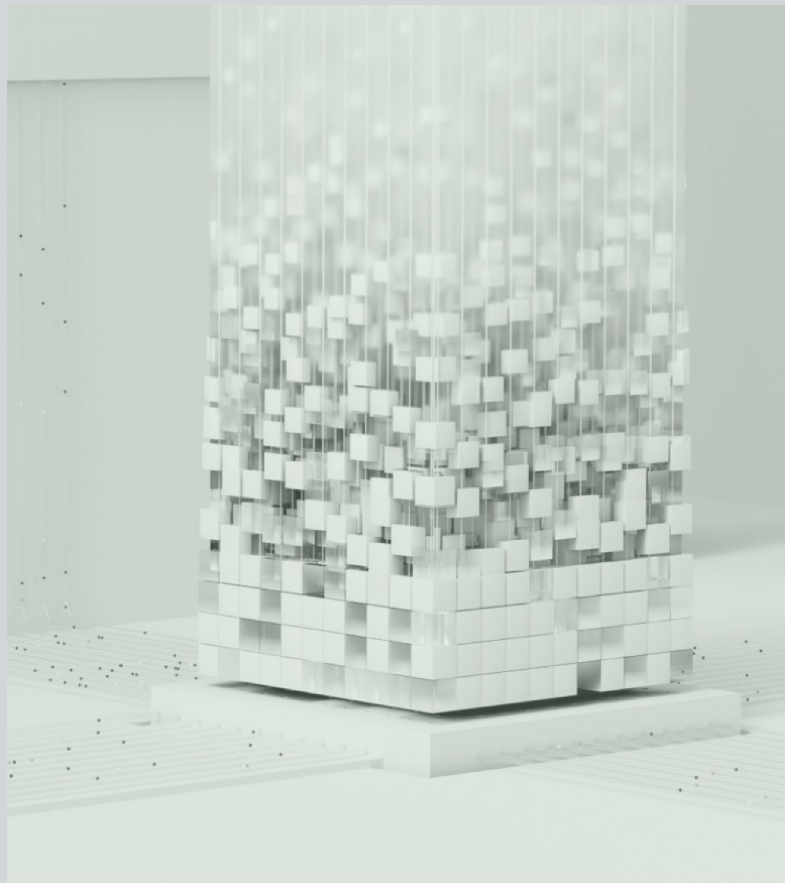
The image shows a screenshot of an API client interface. The 'Body' tab is selected, showing a JSON response with an error. The status is '403 Forbidden'. The JSON response is: { "error": { "code": "function_access_restricted", "message": "Your current subscription plan does not support this API function." } }



Lessons Learned

Lessons Learned

- Learned how to use JPA to connect to database
- Frontend using svelte
- JWT, security & login features



Contribution

Omar *Back-End*

- Implemented flight/user logic
- Implemented tokenization
- Framework for secure login
- Code Review

William *Back-End & Database*

- Database operations
- Database models
- Flight service interface
- Code review
- Documentation

Rianna *Back-End & Database*

- Service layer interaction
- Database operations
- Unit testing

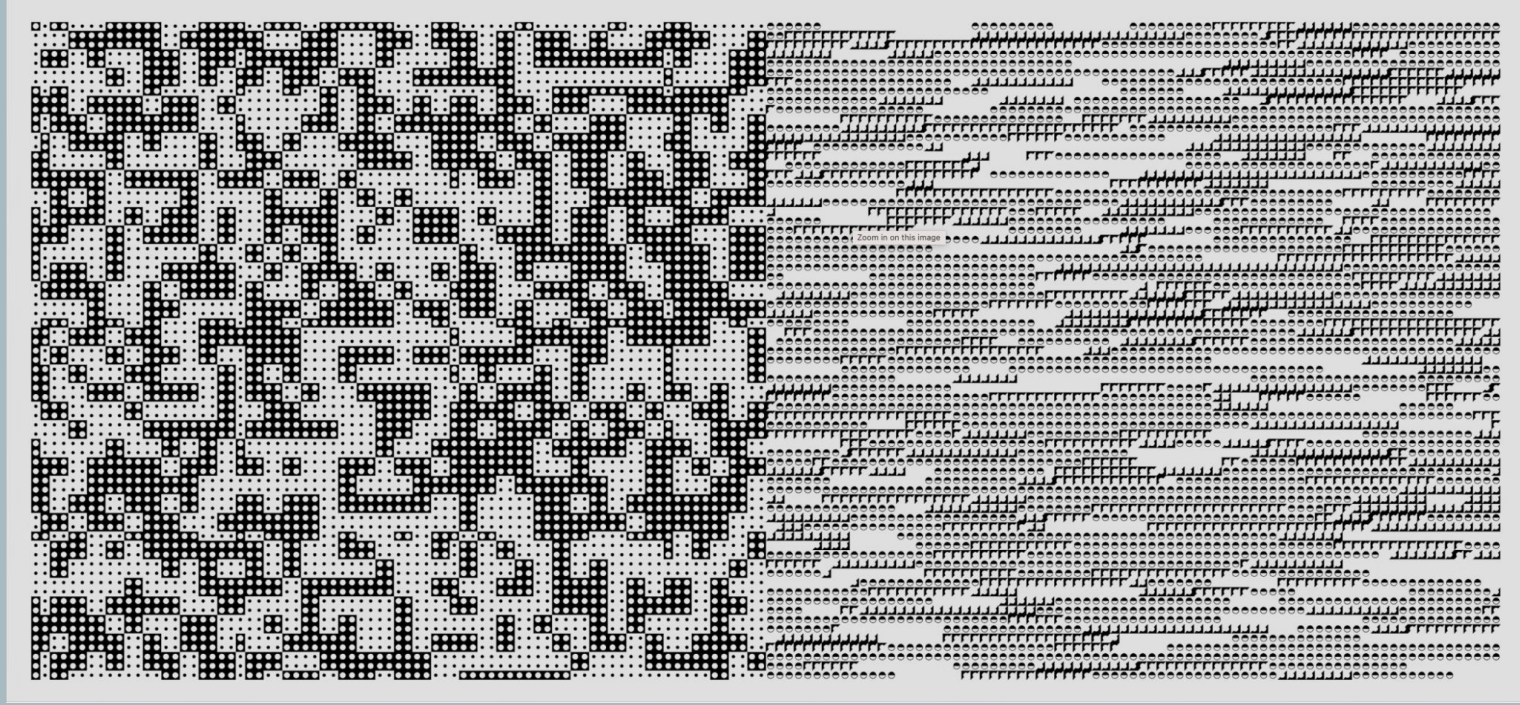
Kyle *Back End, Database, & Frontend*

- Database operations
- Database models
- Implemented live view map
- Frontend design & functionality
- Various backend work

Theebika *Front-End & Back-End*

- Developed front-end, search bar, functions, login, register
- Designed UI using Tailwind CSS
- Unit testing

Q&A



Thank You

Rianna Armour | Theebika Iyer | Omar Usmani | Kyle Yannelli | William Chien

Team 4